

OGC2026 Technical Report

Abstract. This report presents a heuristic for a block scheduling problem that simultaneously involves temporal constraints and geometric constraints on multilayer polygons. For each block, the problem requires determining its bay, orientation, coordinates, entry time, and exit time; in particular, crane accessibility for entry and exit strongly couples placement and time. The core of the proposed method consists of abstract preoptimization, which relaxes the exact geometric constraints into time-indexed area-capacity constraints, and rolling-horizon optimization following the admission order obtained from the abstract solution. In each horizon, a feasible solution is constructed by greedy insertion that integrates a spatial sweep with forbidden entry-time intervals, and is improved by parallel simulated annealing with several neighborhoods, including large reconstruction that removes and reinserts multiple blocks. Because it is difficult to move from a dense packing to a structurally different arrangement using local changes alone, the method emphasizes obtaining a good admission order in advance and optimizing every block at least once while it is near the end of the partial schedule.

1 Introduction

1.1 Challenge Problem & Main Difficulties

For every block i , we determine its assigned bay b_i , orientation o_i , coordinates (x_i, y_i) , entry time e_i , and exit time q_i . The temporal constraints are

$$e_i \geq R_i, \quad q_i - e_i \geq P_i. \quad (1)$$

Blocks occupying the same bay at the same time must not overlap at layers of the same height. Moreover, during an ENTRY or EXIT operation, vertically moving the target block must not collide with layers of other blocks at the same or a greater height than each target layer. Consequently, even a statically non-overlapping placement may be infeasible for crane operations.

The objective is the weighted sum of total tardiness Z_1 , workload imbalance among bays Z_2 , and bay-preference penalty Z_3 :

$$F = w_1 Z_1 + w_2 Z_2 + w_3 Z_3. \quad (2)$$

In particular,

$$Z_1 = \sum_i \max(0, q_i - D_i), \quad Z_3 = \sum_i (S_i^{\max} - S_{i,b_i}), \quad (3)$$

so Z_1 and Z_3 decompose into block-wise contributions. For Z_2 , each bay's total workload is normalized by the ratio of the mean bay area to that bay's area

before taking the maximum difference between bays. Whether these values can be realized, however, depends on the placements and residence intervals of other blocks.

The primary difficulty is the rigidity of the search space. Once a bay becomes densely packed, the space created by moving a single block is often insufficient to improve another block; changing the packing structure requires coordinated changes to several blocks. Furthermore, moving an early block changes its collision relationships and crane ordering with many later blocks. Search performance therefore depends strongly on the initial solution, especially the order in which blocks are admitted to the geometric schedule.

1.2 Contributions

The main contributions of this report are as follows.

- We introduce abstract preoptimization that relaxes multilayer polygon constraints into time-indexed area-capacity constraints and obtains an admission order from the bay assignment and entry time of every block.
- We construct a rolling-horizon framework that expands partial problems according to the abstract admission order, allowing every block to be optimized once while it is at the flexible end of the partial schedule.
- We develop greedy insertion that jointly searches space and time by combining an x -direction sweep for multilayer collisions, based on Minkowski differences, with forbidden entry-time intervals induced by crane constraints.
- We combine small neighborhoods with large reconstruction, which structurally selects and reinserts multiple related blocks, within parallel simulated annealing.

2 Algorithm Description

2.1 Overall Algorithm Framework

Key findings and design rationale. The first key observation concerns a conflict between tardiness and packability. Every unit of block tardiness has the same weight in Z_1 , regardless of the size of the block. Given one large block and several small blocks with similar ENTRY/EXIT conditions, processing the small blocks first can therefore reduce total tardiness. Geometrically, however, a large block is easier to insert while ample free space remains, and inserting it first may produce a better packing. Consequently, an order favorable for Z_1 does not necessarily coincide with an order favorable for geometric feasibility. A simple release-time or due-date order cannot capture this trade-off together with processing times, bay capacities, bay preferences, and competition among blocks.

The second observation is that the geometric constraints cause strong lock-in. Once a schedule has been packed densely, moving to a substantially different packing by changing one block at a time is difficult. Moreover, moving a block

changes its temporal and crane-obstruction relationships with blocks scheduled after it. A block near the beginning of a completed schedule is therefore constrained by many subsequent blocks, whereas a block near the end has few or no subsequent blocks and is much easier to relocate. These properties make both the initial admission order and the stage at which each block is optimized important.

We address the admission-order problem by abstract preoptimization. Exact polygon constraints are relaxed into time-indexed area-capacity constraints, and the relaxed problem is solved over all blocks to obtain a bay b_i and a desirable entry time $\hat{e}_i$ for every block. Sorting by $\hat{e}_i$ provides an admission order that reflects global competition for capacity and the resulting tardiness, rather than relying on a local priority such as release time or due date. The purpose of this stage is not to determine final coordinates, but to predict an effective order in which the exact geometric schedule should be constructed.

We then convert this global order into an exact schedule by rolling-horizon optimization. Each horizon admits only a limited number of new blocks and optimizes a partial problem. This decomposition is well aligned with the original objective because its dominant terms in many instances are block-wise additive:

$$Z_1 = \sum_i T_i, \quad Z_3 = \sum_i (S_i^{\max} - S_{i,b_i}). \quad (4)$$

Improving these terms for admitted blocks therefore directly contributes to improving the corresponding terms of the full problem. In addition, every block is once newly admitted near the tail of a partial schedule, where it can be moved with relatively few dependencies on subsequent blocks. Limiting the number of admitted blocks also reduces the cost of neighborhood evaluation and permits more search iterations than repeatedly optimizing the complete problem. Earlier blocks are not frozen: all admitted blocks remain movable, so decisions can still be corrected when later blocks are introduced.

Finally, rolling-horizon optimization alone does not eliminate geometric lock-in within each partial problem. In a dense schedule, single-block neighborhoods such as Shift and Move often cannot cross infeasible intermediate configurations. Large reconstruction therefore removes several spatially, temporally, or objectively related blocks and reinserts them in a new order, allowing the packing structure to change in one neighborhood move. The three main ideas thus have complementary roles: abstract preoptimization determines a globally informed admission order, rolling-horizon optimization gives every block a relatively flexible stage for exact optimization, and large reconstruction enables structural changes after blocks have been packed.

Solution procedure. The solver first precomputes geometric information and candidate relations, and then solves the area-capacity relaxation to obtain $(\hat{b}_i, \hat{e}_i)$ for every block. Blocks are admitted in lexicographic order of $(\hat{e}_i, -V_i, i)$, where V_i is the maximum footprint area times the processing time. Thus, among blocks with the same abstract entry time, larger blocks are admitted first. The order

Algorithm 1 Overall solution procedure

Require: problem instance, horizon size H , global deadline τ
Ensure: feasible geometric schedule S

```

1:  $\{(\hat{b}_i, \hat{e}_i)\}_i \leftarrow \text{ABSTRACTPREOPTIMIZATION}$ 
2:  $\pi \leftarrow$  blocks sorted lexicographically by  $(\hat{e}_i, -V_i, i)$ 
3:  $\mathcal{H} \leftarrow \text{EVENLYPARTITION}(\pi, \lceil N/H \rceil)$ 
4:  $\{S_r\}_r \leftarrow \{\emptyset\}_r$ 
5: for each horizon  $h \in \mathcal{H}$  do
6:   for each worker  $r$  in parallel do
7:      $S_r \leftarrow \text{EXTENDSCHEDULE}(S_r, h)$ 
8:   end for
9:   if  $h$  is the final horizon then
10:     $w'_2 \leftarrow w_2$ 
11:   else
12:     $w'_2 \leftarrow 0$ 
13:   end if
14:    $\tau_h \leftarrow \text{ALLOCATEREMAININGTIME}(h, \mathcal{H}, \tau)$ 
15:    $\{S_r\}_r \leftarrow \text{PARALLELSA}(\{S_r\}_r, w_1, w'_2, w_3, \tau_h)$ 
16: end for
17: return the best  $S_r$ 

```

is evenly partitioned into $\lceil N/H \rceil$ horizons. At each horizon, every worker extends its own solution by repeatedly removing a small random number of admitted blocks, mixing them with the new blocks, and greedily reinserting them in random order; the best extension found within the construction-time budget is retained. All admitted blocks are then improved; the final horizon uses the complete objective containing Z_1 , Z_2 , and Z_3 . Algorithm 1 summarizes this procedure, focusing on how the abstract schedule is converted into an exact geometric schedule. The abstract entry times determine the admission order and provide a reference solution for insertion, but neither the bay nor the entry time is fixed in the geometric optimization. In particular, $\hat{e}_i$ is not imposed as a lower bound on the exact entry time.

2.2 Key Ideas that Succeeded

Abstract preoptimization. For orientation o of block i , let A_{io}^{union} be the area of the union of all layers, A_{io}^{bbox} be the area of its bounding box, and L_{io} be the perimeter of the union. If orientation o fits in bay j , its approximate occupancy is

$$a_{ijo} = A_{io}^{\text{union}} + \alpha (A_{io}^{\text{bbox}} - A_{io}^{\text{union}}) + \beta L_{io}. \quad (5)$$

We define a_{ij} as the minimum value over orientations that fit in bay j . The first term represents the actual occupied area, the second approximates unusable space around concave or elongated shapes, and the third compensates for packing difficulty caused by a complex boundary.

Let C_j be the effective capacity of bay j :

$$C_j = \max \left\{ W_j H_j, \max_{i: a_{ij} \text{ is defined}} a_{ij} \right\}. \quad (6)$$

The lower bound by the largest approximate occupancy avoids an abstract capacity that excludes a fitting block by itself. The abstract schedule must satisfy

$$\sum_{i: \hat{e}_i \leq t < \hat{e}_i + P_i, \hat{b}_i = j} a_{ij} \leq C_j \quad (7)$$

at every time. An abstract state stores only $(\hat{b}_i, \hat{e}_i)$ for each block and does not store its exact orientation or coordinates. Its objective consists of Z_1 , Z_2 , and Z_3 evaluated on the abstract schedule.

Initial states are built greedily from multiple randomized block orders. Each order combines workload, the product of area and processing time, the spread of bay preferences, the limit time defined as due date minus processing time, slack, and a random component. For each block, the earliest capacity-feasible time is found in every eligible bay, and the candidate minimizing the incremental Z_1 , Z_2 , and Z_3 is selected. The state is then improved by simulated annealing using Relocate and Swap. Relocate selects a block uniformly, visits bays in random order, and tests its release time, its limit time, and a random entry time. Swap exchanges the bay assignments of two blocks and either retains or exchanges their entry times.

Rolling-horizon geometric optimization. Blocks are sorted by $(\hat{e}_i, -V_i, i)$, and the order is evenly divided into $\lceil N/H \rceil$ horizons. At each horizon, every worker carries its own solution forward. Schedule extension repeatedly removes k existing blocks, combines them with the newly admitted blocks, and greedily reinserts the combined set in random order. Trials continue for 5% of the horizon budget, capped at five seconds, and the best extension is retained. Simulated annealing is then applied not only to the new blocks but to all blocks admitted so far. Unlike a freezing-type rolling horizon, earlier decisions can therefore be revised in response to later admissions. This follows the general rolling-horizon principle of solving a sequence of restricted scheduling problems[3], while retaining all previous blocks as movable variables in our geometric search.

Because Z_2 is not meaningful until the bay assignments of all blocks are present, its weight is set to zero in intermediate horizons. The official w_2 is enabled only in the final horizon. Search time is allocated with weights proportional to a power of the number of admitted blocks, reserving more time for the larger late-stage problems.

Specifically, let N_h denote the cumulative number of admitted blocks in horizon h , and let γ be a parameter. Ignoring time already consumed by construction and other overhead, the nominal allocation is

$$\tau_h = \tau_{\text{total}} \frac{N_h^\gamma}{\sum_q N_q^\gamma}. \quad (8)$$

Algorithm 2 Greedy geometric insertion of block i

Require: partial schedule S , block i , reference entry time $\hat{e}_i$

```

1:  $C \leftarrow \emptyset$  and  $F^* \leftarrow +\infty$ 
2: for each eligible bay  $j$  whose lower bound does not exceed  $F^*$  do
3:   for each orientation  $o$  and shuffled feasible row  $y$  do
4:      $E \leftarrow \text{BUILDXEVENTS}(S, i, j, o, y)$ 
5:     for each candidate  $x$  in the event sweep over  $E$  do
6:       update directional collision states at  $x$ 
7:        $\mathcal{I} \leftarrow$  corresponding forbidden entry-time intervals
8:        $e \leftarrow \text{FIRSTFEASIBLETIME}(\mathcal{I}, R_i)$ 
9:       if  $e$  exists then
10:        update  $F^*$  and the ordered set  $C$  of up to four best tied candidates
11:       end if
12:     end for
13:   end for
14: end for
15: return a probabilistic choice from  $C$ 

```

In the implementation, this ratio is recomputed from the remaining horizons and the remaining time at the beginning of every horizon, so earlier overruns do not invalidate the global deadline. We use $\gamma > 1$ because later horizons contain more interacting blocks, have more expensive neighborhood evaluations, and are closer to the final complete objective. At the same time, early horizons still receive enough time to establish a packing structure before additional blocks reduce the available space.

Greedy geometric insertion and collision detection. Greedy insertion enumerates bays, orientations, and integer y coordinates, while sweeping in the x direction through collision events. A candidate s is compared primarily by

$$\Delta F(s) = w_1 T_i(s) + w_2 \Delta Z_2(s) + w_3 (S_i^{\max} - S_{i, b_i(s)}). \quad (9)$$

Among candidates tied with the best objective value, an earlier entry time and placement near a bay corner are preferred. Workers 0–3 use bottom-left, bottom-right, top-left, and top-right as their respective primary anchors. Each insertion switches to another corner with probability 0.2, providing parallel diversity. The feasible y coordinates are shuffled; after the first feasible row is found, only ten additional rows are examined. Among candidates tied at the best objective value, at most the top four are retained and selected probabilistically, choosing from the front with probability 0.7.

Algorithm 2 outlines the insertion procedure. Rather than testing every (x, e_i) , the sweep updates collision states at x -interval endpoints and converts them into forbidden entry-time intervals.

For collision detection, each nonconvex layer is decomposed into convex polygons by ear clipping. For two convex polygons A and B , a collision occurs when

their relative displacement $(\Delta x, \Delta y)$ belongs to the Minkowski difference[2]

$$A \oplus (-B) = \{a - b \mid a \in A, b \in B\}. \quad (10)$$

This region is horizontally sliced at every integer Δy and converted into intervals of forbidden integer Δx . A small margin prevents missed collisions due to numerical errors, and overlapping or adjacent intervals in the same row are merged. For crane constraints, forbidden regions are united for every moving layer k and fixed layer $l \geq k$. Because crane obstruction is directional, the two directions of every orientation pair are stored separately. The collision grid is lazily constructed on first use and then cached.

Joint spatial-temporal sweep. The distinguishing feature of the insertion procedure is that spatial and temporal searches are not separated. For every existing block, the algorithm separately tracks whether the new block is obstructed by the existing block and whether the existing block is obstructed by the new block. Endpoints of forbidden x intervals are treated as events, and these two directional states are updated during the x sweep. From the current state and the residence interval of the existing block, the forbidden entry times of the new block can be represented by at most two intervals. All active forbidden intervals at the current coordinates are merged to find the earliest feasible entry time in the search range. Hence, the full schedule need not be revalidated at every coordinate and every day, and the running time does not directly depend on the time horizon.

Let n_j be the number of blocks already placed in bay j , let W_j and H_j be its width and height, and let O_i be the number of orientations of the inserted block. Treating the number of collision intervals per block pair as constant, event generation and counting sort for one (o, y) require $O(n_j + W_j)$ time. A conservative bound including scans of forbidden time intervals at candidate positions is $O(W_j + n_j^2)$. The approximate worst-case complexity of inserting one block is therefore

$$O\left(\sum_j O_i H_j (W_j + n_j^2)\right). \quad (11)$$

Simulated annealing and large reconstruction. The local search is based on simulated annealing[1] and uses four neighborhood types: Large Reconstruction, Shift, Move, and Rotate. Shift samples a nearby displacement (default range $[-5, 0]$, with its sign determined by the anchor) and then scans to redetermine x , y , and the entry time while retaining the bay and orientation. Rotate changes to a geometrically similar orientation precomputed by bounding-box IoU. Move selects a block uniformly at random and redetermines its bay, orientation, position, and entry time by greedy insertion, restricting the new entry time to be no later than its current value.

Large reconstruction is a destroy-and-repair large neighborhood. It selects multiple seeds using three methods: badness, fluidity, and random selection.

Badness evaluates tardiness, preference penalty, and membership in the bay with the heaviest workload. Fluidity favors blocks with large slack and a small spread of bay preferences, which are easier to move. Blocks in the same bay as each seed are ordered by

$$d(i, s) = \lambda_x |x_i - x_s|^p + \lambda_y |y_i - y_s|^p + \lambda_t |e_i - e_s|^p, \quad p \sim U[2, 4], \quad (12)$$

where both the exponent and weights are sampled for each trial. After the first seed, seed candidates are biased toward entry times close to those of the existing seeds, grouping reconstruction by entry-time base. Nearby blocks are removed together. Multiple seeds make it possible to change several local structures in one move. The reinsertion order combines volume, preference spread, limit-time urgency, slack, current penalty, and randomness; workload is used only as a tie-breaker. The weights are randomized for every attempt, allowing the search to revisit the trade-off between tardiness and packability without being restricted to a fixed size or due-date order. Moreover, the current SA acceptance threshold is inverted into a per-block upper bound on admissible tardiness, pruning insertion candidates that cannot be accepted.

Simulated annealing is executed by multiple workers in parallel. Each worker preserves its own state across horizons, updates a shared best state, and periodically imports a sufficiently better shared solution. At each horizon, the initial temperature is proportional to the initial score per admitted block, and the temperature follows an exponential schedule from T_0 to T_1 ; the default multipliers are 0.2 and 0.02, respectively. This schedule is reset for each horizon.

2.3 Key Ideas that Failed

Beam-search reconstruction attempted to reuse block-insertion results relative to the same base schedule across multiple search states, thereby exploring reinsertion orders at lower computational cost. However, exploring diverse placement states proved more important than reusing insertion results, and the method was discarded.

Multi-stage optimization, which optimized tardiness, preference, and workload imbalance in separate stages, did not show a significant advantage over the standard search. Reheating by returning to a best state and applying a kick, hash-based tabu, gradual introduction of w_2 , optimization of same-time operation order, and temporal changes based on exchanging exit times were also tested and discarded. We additionally tested adaptive annealing derived from observed objective differences and a blocker-aware neighborhood that explicitly removed crane blockers. These variants either added overhead or lacked consistent improvement across instances.

2.4 Further Improvement Plan

For abstract preoptimization, we plan to improve the approximate occupancy model so that the gap between abstract schedules and exact placements is reduced. For rolling-horizon optimization, horizon size and search-time allocation

could be adapted to each instance. For large reconstruction, blocker dependencies could be used more directly in selecting blocks for removal.

For simulated annealing, in addition to revisiting adaptive temperature control based on objective differences and acceptance ratios, we plan to introduce a more systematic parallel scheme. Workers would specialize in different temperature ranges or neighborhood compositions and periodically exchange states, explicitly separating structural changes by high-temperature workers from local improvement by low-temperature workers.

3 Implementation Details

The solver is implemented in Rust. Convex decompositions, bounding boxes, feasible placement ranges in bays, orientation neighbors based on bounding-box IoU, preference-ordered bay lists, orientations ordered by bounding-box area, preference penalties and spreads, bay workload scales, and maximum footprint areas are precomputed. Collision grids for block and orientation pairs are built on demand and shared throughout the subsequent search. The placement sweep processes only endpoints of forbidden x intervals and uses counting sort and bitsets to maintain active time intervals.

The time limit is divided between abstract preoptimization and rolling horizon optimization. In the latter phase, the deadline of each horizon is computed from the sizes of the remaining horizons. The current shared best solution is emitted periodically during the search. EXIT operations are added to the output before ENTRY operations, ensuring that every EXIT precedes every ENTRY at the same time.

4 Computational Results

All methods were evaluated once on the 40 instances in the final training set. Experiments were conducted on an Apple M1 Pro with eight CPU cores and 32 GB of memory. Each run used four worker threads and a time limit of 300 seconds. “w/o preoptimization” replaces abstract preoptimization with a simple due-date admission order, while “w/o rolling horizon” admits all blocks in a single horizon. All runs produced feasible solutions. Table 1 reports the objective components aggregated over all instances, and Table 2 reports the total objective for each instance together with its numbers of bays and blocks.

Table 1. Aggregated objective values on the training dataset. Lower is better.

Metric	Solution w/o preoptimization w/o rolling horizon		
Z_1	412,209,673	455,157,137	452,373,802
Z_2	534,593	520,229	428,491
Z_3	35,526,753	38,580,980	43,548,790
Total	448,271,019	494,258,346	496,351,083

Table 2. Case-wise total objective values. The best value in each row is bold.

Case	Bays	Blocks	Solution	w/o preopt.	w/o RH
1	3	150	200,600	254,893	294,493
2	3	250	51,514,630	56,503,516	56,822,966
3	4	200	3,861,664	3,727,803	3,827,482
4	4	250	1,495,165	1,284,783	1,842,738
5	2	150	6,128,553	6,379,911	6,538,337
6	4	250	4,453,575	4,395,833	4,595,350
7	3	150	344,892	294,701	286,315
8	4	200	57,275	60,110	64,021
9	3	200	4,787,490	4,515,461	5,387,753
10	2	150	2,514,378	3,196,467	3,362,278
11	4	250	6,158,320	6,153,697	6,731,431
12	5	300	108,905	109,140	109,631
13	2	300	65,583,089	71,348,407	69,348,586
14	4	250	509,993	580,705	590,104
15	3	250	9,117,871	10,285,613	10,778,146
16	5	300	844,749	1,242,098	2,748,614
17	3	250	4,166,279	4,582,154	4,792,621
18	3	200	7,760,385	8,633,235	8,996,725
19	3	200	14,253,229	15,638,146	16,154,618
20	3	250	7,226,052	8,020,782	9,047,633
21	2	150	601,313	629,425	689,846
22	3	150	1,698	1,964	1,964
23	5	300	10,860,631	11,985,764	12,359,963
24	3	150	1,401,057	1,702,012	2,014,100
25	2	300	65,341,045	74,668,806	72,491,167
26	2	300	22,713,804	26,057,775	24,568,661
27	4	200	113,267	137,490	181,368
28	4	250	12,835,222	13,600,998	14,918,839
29	4	200	172,024	182,248	205,686
30	5	300	1,290,398	1,261,997	1,707,717
31	3	200	2,514,690	3,028,944	3,303,448
32	4	200	545,663	603,104	603,576
33	2	150	452,702	372,622	987,223
34	5	300	128,823	127,209	148,527
35	4	250	81,715	83,496	140,982
36	2	300	69,984,887	78,781,616	75,387,709
37	2	150	14,170,930	15,999,477	15,299,472
38	4	200	6,466,785	7,300,523	6,973,239
39	2	150	41,480,168	43,320,130	44,791,617
40	2	300	6,027,103	7,205,291	7,256,137

As shown in Table 1, removing preoptimization increased the total objective by 10.26%, including increases of 10.42% in Z_1 and 8.60% in Z_3 . This result is consistent with the intended role of the relaxed area-capacity model: determining an admission order that accounts for competition among blocks more effectively than a simple due-date order.

Table 1 also shows that removing the rolling horizon increased the total objective by 10.73%, with increases of 9.74% in Z_1 and 22.58% in Z_3 . This is consistent with the intended effects of the rolling horizon: solving smaller subproblems allows more iterations, and every block is temporarily placed near the tail of a partial schedule, where it can be moved without disturbing many subsequent blocks. The particularly large change in Z_3 is also consistent with the interpretation that this additional flexibility helps select better bay assignments.

In Table 1, both ablations obtained smaller Z_2 values, but these gains were outweighed by their deterioration in the dominant Z_1 and Z_3 terms. The complete method achieved the best total objective in 31 of the 40 rows of Table 2, compared with eight instances for w/o preoptimization and one for w/o rolling horizon. Thus, abstract preoptimization and rolling-horizon optimization play

complementary roles: the former provides an effective global insertion order, while the latter converts that order into high-quality geometric placements.

5 Team Members' Contributions

This section is not applicable because the team consists of a single member.

6 AI Use Declaration

In this project, primarily Codex with GPT 5.6 Sol High was used to assist with implementing algorithms conceived by the author. For this technical report, it was used to organize the existing implementation and prepare the draft. The algorithmic ideas and design decisions were made by the author, who also reviewed, validated, and revised the generated code and text.

References

1. S. Kirkpatrick, C. D. Gelatt, Jr., and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
2. P. K. Ghosh, "A Unified Computational Framework for Minkowski Operations," *Computers & Graphics*, vol. 17, no. 4, pp. 357–378, 1993.
3. W. Ouyang, G. Dalle Ave, I. Harjunoski, and L. Imsland, "A Rolling Horizon Approach for Scheduling of Multiproduct Batch Production and Maintenance Using Generalized Disjunctive Programming Models," *Computers & Chemical Engineering*, vol. 148, article 107268, 2021.